

MCPShield LocalStack Guide

Complete Guide to the Mock AWS Environment for Build with AI OAU 2026

Author: Olumayowa Akinkuehinmi

Event: Build with AI OAU 2026 — Cybersecurity Breakout

Date: July 25, 2026

Repository: <https://github.com/akintunero/mcpshield>

Table of Contents

1. [What is LocalStack?](#)
 2. [Why LocalStack for This Workshop?](#)
 3. [Installation](#)
 4. [Starting LocalStack](#)
 5. [Verifying LocalStack is Running](#)
 6. [Understanding LocalStack Services](#)
 7. [How MCPShield Connects to LocalStack](#)
 8. [Provisioning Resources with the Bootstrap Script](#)
 9. [The 15 Vulnerable Resources — What Gets Created](#)
 10. [Manually Interacting with LocalStack](#)
 11. [How the Scanner Works](#)
 12. [How Remediation Works](#)
 13. [Common LocalStack Commands](#)
 14. [Troubleshooting LocalStack](#)
 15. [Advanced: Custom LocalStack Configuration](#)
 16. [Advanced: Multi-Region Setup](#)
 17. [Advanced: Persistent State](#)
 18. [Advanced: Lambda Functions in LocalStack](#)
 19. [Resetting the Environment](#)
 20. [Frequently Asked Questions](#)
 21. [Quick Reference](#)
-

1. What is LocalStack?

LocalStack is a fully functional local AWS cloud stack. It emulates AWS services on your local machine, allowing you to develop and test cloud applications without connecting to a real AWS account.

Think of it as **AWS on your laptop** — but free, offline, and completely isolated.

What LocalStack Emulates

LocalStack provides mock implementations of 15+ AWS services that respond with realistic API responses. For MCPShield, we use these services:

AWS Service	LocalStack Support	What MCPShield Uses It For
S3	✔ Full	Creating vulnerable buckets, checking public access, encryption, versioning, logging
IAM	✔ Full	Creating users with admin policies, managing access keys, password policies
EC2	✔ Full	Creating security groups with open ports
CloudTrail	✔ Full	Creating trails, disabling logging
Lambda	✔ Full	Creating functions for resource diversity
SQS	✔ Full	Creating queues for resource diversity
SNS	✔ Full	Creating topics for resource diversity
Secrets Manager	✔ Full	Creating secrets for resource diversity
SSM Parameter Store	✔ Full	Creating parameters with empty descriptions
DynamoDB	✔ Full	Creating tables for resource diversity
CloudWatch	✔ Full	Creating alarms for resource diversity

What LocalStack Does NOT Emulate

Some AWS services are not available or have limited support:

- **AWS Organizations** — Not available in the free version
- **AWS WAF** — Limited support
- **AWS Shield** — Not available
- **Amazon Inspector** — Not available
- **Amazon GuardDuty** — Not available
- **AWS Config** — Limited support (available in LocalStack Pro)

For MCPShield, these limitations don't matter — the 11 supported services are enough for a comprehensive security workshop.

2. Why LocalStack for This Workshop?

The Problem with Real AWS

If MCPShield used real AWS accounts for this workshop:

1. **Cost** — Every API call costs money. Scanning 11 services across 60 participants would generate significant AWS bills.
2. **Risk** — The workshop creates intentionally vulnerable resources (public S3 buckets, open security groups). On a real account, these could be exploited.
3. **Complexity** — Every participant would need an AWS account, IAM credentials, and appropriate permissions.
4. **Reset** — After the workshop, participants would need to manually clean up all resources.

The LocalStack Advantage

Aspect	Real AWS	LocalStack
Cost	Pay per API call	Free
Safety	Real security risk	Isolated — no risk
Setup	AWS account + IAM	pip install localstack
Speed	Network latency	Instant (local)
Reset	Manual cleanup	Ctrl+C, restart
Offline	Requires internet	Works offline
Repeatable	State changes persist	Fresh every time

Workshop-Specific Benefits

For a 60-minute workshop with 60 participants:

1. **No AWS account needed** — Participants don't need credit cards or sign up for AWS
 2. **No internet dependency** — Once installed, LocalStack works completely offline
 3. **Instant reset** — If something breaks, restart LocalStack and re-bootstrap in seconds
 4. **Identical environments** — Every participant gets exactly the same vulnerable resources
 5. **Safe exploration** — Participants can experiment freely without breaking anything real
-

3. Installation

Prerequisites

Before installing LocalStack, ensure you have:

- **Python 3.9+** — LocalStack is a Python application
- **pip** — Python package manager
- **Docker** — Recommended but optional (LocalStack can run in Docker or directly)

Check Python Version

```
python3 --version
# Expected: Python 3.9.x or higher

pip3 --version
# Expected: pip 21.x or higher
```

Install via pip (Recommended)

This is the simplest and most reliable method for the workshop:

```
# Install LocalStack
pip3 install localstack

# Verify installation
localstack --version
# Expected: localstack 4.x.x
```

Troubleshooting pip installation:

```
# If you get permission errors:
pip3 install --user localstack

# On macOS, if you get "externally managed environment" errors:
pip3 install --break-system-packages localstack

# Or use a virtual environment:
python3 -m venv mcpshield-env
source mcpshield-env/bin/activate
pip install localstack
```

Install via Docker (Alternative)

If you prefer Docker:

```
# Pull the LocalStack image
docker pull localstack/localstack:latest

# Verify
docker images | grep localstack
```

Install via Homebrew (macOS)

```
brew install localstack/tap/localstack-cli
localstack --version
```

Verify Installation

After installation, verify LocalStack is accessible:

```
localstack --help
# Should show the help menu with available commands
```

4. Starting LocalStack

Method 1: Default Start (Recommended for Workshop)

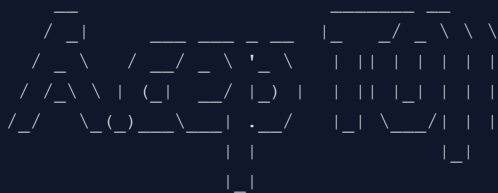
The simplest way to start LocalStack:

```
localstack start -d
```

What this does:

- Starts LocalStack as a background daemon (`-d` flag)
- Listens on `http://localhost:4566`
- Enables all available services
- Runs until you stop it

Expected output:

The image shows the LocalStack logo, which is a stylized 'A' shape composed of various AWS service icons and names arranged in a grid-like pattern.

```
LocalStack version: 4.x.x
LocalStack Docker container started: localstack-main
```

Method 2: Docker Run

```
docker run --rm -it \
  -p 4566:4566 \
  -p 4510-4559:4510-4559 \
  -e SERVICES=s3,iam,ec2,cloudtrail,lambda,sqs,sns,secretsmanager,ssm,dynamodb,cloudwatch \
  localstack/localstack
```

Method 3: Foreground Mode (For Debugging)

```
localstack start
```

This runs LocalStack in the foreground. You'll see all logs in real time. Press `Ctrl+C` to stop.

What Happens When LocalStack Starts

1. LocalStack initializes its internal state
2. It starts listening on port 4566
3. It enables all configured services
4. It checks for any existing state from previous runs
5. It becomes ready to accept API requests

The startup takes about 5-10 seconds on most machines.

Stopping LocalStack

```
localstack stop
```

Or if started with Docker:

```
docker stop localstack-main
```

Restarting LocalStack

```
localstack stop  
localstack start -d
```

Or for a complete reset:

```
localstack stop  
localstack start -d # Fresh state
```

5. Verifying LocalStack is Running

Health Check Endpoint

The most reliable way to verify LocalStack is running:

```
curl http://localhost:4566/_localstack/health
```

Expected response:

```
{
  "services": {
    "s3": "available",
    "iam": "available",
    "ec2": "available",
    "cloudtrail": "available",
    "lambda": "available",
    "sqs": "available",
    "sns": "available",
    "secretsmanager": "available",
    "ssm": "available",
    "dynamodb": "available",
    "cloudwatch": "running"
  },
  "features": {
    "persistence": "off",
    "initScripts": "initialized"
  },
  "status": "running"
}
```

What to look for:

- `"status": "running"` — LocalStack is operational
- Individual services show `"available"` or `"running"`
- If a service shows `"starting"`, wait a few seconds and try again

Quick Connectivity Test

```
# Check if port 4566 is listening
curl -s -o /dev/null -w "%{http_code}" http://localhost:4566
# Expected: 200 (or 404 — both mean LocalStack is responding)
```

AWS CLI Health Check

```
# List S3 buckets (should return empty list, not an error)
aws --endpoint-url=http://localhost:4566 s3 ls
# Expected: (no output — no buckets yet)

# List IAM users
aws --endpoint-url=http://localhost:4566 iam list-users
# Expected: { "Users": [] }
```

MCPShield Health Check

Once MCPShield is running, you can also check LocalStack connectivity through MCP:

```
curl http://localhost:7801/health
```

Look for:

```
{
  "status": "healthy",
  "localstack": { "connected": true, "endpoint": "http://localhost:4566" }
}
```

What If Services Are Not Available?

Some services may show as `"stopped"` if they were not enabled when LocalStack started. This is normal if you're using a specific service configuration. For the workshop, all 11 services should be available.

If a service shows `"stopped"` :

```
# Restart LocalStack with all services enabled
localstack stop
localstack start -d
```

6. Understanding LocalStack Services

How Services Work

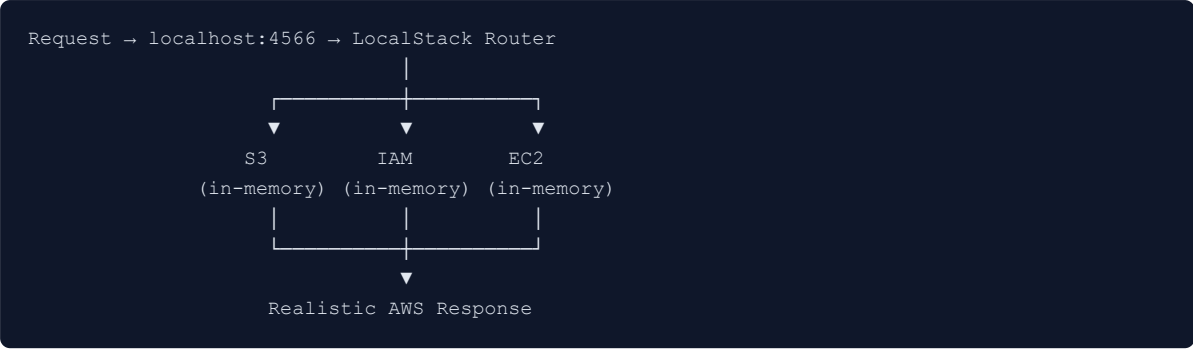
LocalStack intercepts AWS API calls at the network level. When you (or MCPShield) send an AWS API request to `http://localhost:4566`, LocalStack:

1. **Receives** the HTTP request
2. **Parses** the AWS API action from the headers
3. **Routes** it to the appropriate mock service
4. **Processes** the request against its in-memory state
5. **Returns** a realistic AWS-style response

This means:

- You use the exact same AWS SDK calls as real AWS
- You get back the same JSON response structures
- Authentication is accepted (any credentials work)
- No data leaves your machine

Service Lifecycle



How LocalStack Differs from Real AWS

Aspect	Real AWS	LocalStack
Authentication	Real IAM credentials	Any credentials work
Latency	50-500ms per call	<5ms per call
Data persistence	Permanent	In-memory (lost on restart)
Service limits	Real AWS limits apply	No limits
DNS resolution	Route53	Localhost only
IAM policies	Enforced	Not enforced (permissive)
Encryption	KMS-managed	Mocked
CloudWatch metrics	Real metrics	Mocked

Why IAM Policies Are Not Enforced

In LocalStack, authentication is accepted but **authorization is not enforced**. This means:

```
# This works in LocalStack even without admin credentials
aws --endpoint-url=http://localhost:4566 iam attach-user-policy \
  --user-name anyone \
  --policy-arn arn:aws:iam::aws:policy/AdministratorAccess
```

This is actually **good for the workshop** because:

- Participants don't need to configure complex IAM permissions
- All operations are allowed (we control what's vulnerable through provisioning)
- Focus stays on security scanning, not AWS permission troubleshooting

7. How MCPShield Connects to LocalStack

Configuration

MCPShield connects to LocalStack through environment variables in the `.env` file:

```
# .env - LocalStack Configuration
LOCALSTACK_ENDPOINT=http://localhost:4566
AWS_ACCESS_KEY_ID=test
AWS_SECRET_ACCESS_KEY=test
AWS_DEFAULT_REGION=us-east-1
```

How the Connection Works

```
MCPShield Agent
|
▼
MCP Server (apps/mcp-server/)
|
▼
AWS SDK Clients (packages/aws-tools/src/clients.ts)
|
▼ Configuration:
|   endpoint: http://localhost:4566
|   region: us-east-1
|   credentials: { accessKeyId: "test", secretAccessKey: "test" }
|
▼
LocalStack (http://localhost:4566)
```

The clients are created in `packages/aws-tools/src/clients.ts` :

```
import { S3Client } from '@aws-sdk/client-s3';
import { getConfig } from '@mcpshield/config';

export function getClientConfig() {
  const config = getConfig();
  return {
    endpoint: config.aws.endpoint,      // http://localhost:4566
    region: config.aws.region,         // us-east-1
    credentials: {
      accessKeyId: config.aws.accessKeyId,
      secretAccessKey: config.aws.secretAccessKey,
    },
  };
};

export const s3Client = new S3Client({ ...getClientConfig(), forcePathStyle: true });
```

Docker vs. Local

When running MCPShield locally (not in Docker):

```
LOCALSTACK_ENDPOINT=http://localhost:4566
```

When running MCPShield in Docker Compose:

```
LOCALSTACK_ENDPOINT=http://host.docker.internal:4566
```

The `host.docker.internal` hostname allows Docker containers to reach services running on the host machine (where LocalStack runs).

Verifying the Connection

MCPShield's `health` MCP tool checks LocalStack connectivity:

```
curl http://localhost:7801/health
```

Internally, this calls:

```
async function checkLocalStackHealth(): Promise<boolean> {
  try {
    const res = await fetch(`${LOCALSTACK_ENDPOINT}/_localstack/health`);
    const data = await res.json();
    return data.status === 'running';
  } catch {
    return false;
  }
}
```

8. Provisioning Resources with the Bootstrap Script

What Provisioning Means

Provisioning creates the intentionally vulnerable AWS resources that MCPShield will scan. Without provisioning, LocalStack is empty — no buckets, no users, no security groups to scan.

The Bootstrap Script

The bootstrap script (`scripts/bootstrap.sh`) automates the entire setup:

```
#!/usr/bin/env bash
# scripts/bootstrap.sh

set -euo pipefail

# Step 1: Check Docker
# Step 2: Check LocalStack
# Step 3: Build monorepo
# Step 4: Provision vulnerable resources
# Step 5: Validate readiness
```

Running the Bootstrap

```
./scripts/bootstrap.sh
```

Expected output: ``

MCPShield Workshop Setup & Bootstrap

- =====
- 🔗 Target LocalStack Endpoint: <http://localhost:4566>
 - 🔍 Checking Docker status... ✅ Docker is active and running.
 - 🔍 Verifying LocalStack endpoint connectivity... ✅ LocalStack is reachable and healthy.
 - 🔧 Building monorepo workspace packages... ✅ Build completed successfully.
 - ⚡ Provisioning vulnerable resources in LocalStack... ✅ Vulnerable AWS resources successfully provisioned.
 - 📁 Validating MCPShield readiness status... ✓ LocalStack connection: Healthy ✓ AWS provisioning: Provisioned ✓ Monorepo package build: Compiled
 - 🎉 Setup Successful! MCPShield is ready for the workshop!

What the Provision Script Does

The provision script (`scripts/provision.ts`) creates these resources:

```
```typescript
// scripts/provision.ts - Simplified overview

async function main() {
 // 1. S3 - Vulnerable bucket (public, no encryption, no versioning)
 await s3Client.send(new CreateBucketCommand({ Bucket: 'vulnerable-bucket' }));
 await s3Client.send(new PutBucketPolicyCommand({
 Bucket: 'vulnerable-bucket',
 Policy: JSON.stringify({
 Version: '2012-10-17',
 Statement: [{
 Effect: 'Allow',
 Principal: '*',
 Action: 's3:GetObject',
 Resource: 'arn:aws:s3:::vulnerable-bucket/*'
 }]
 })
 }));
 // Also creates a log bucket for logging findings later

 // 2. IAM - Users with various vulnerabilities
 // admin-user: Has AdministratorAccess policy
 // stale-key-user: Has 90+ day old access keys
 // unused-key-user: Has never-used access keys
 // unused-user: Has no activity at all
 // Weak password policy: No complexity requirements

 // 3. EC2 - Security group with SSH (22) and RDP (3389) open to 0.0.0.0/0

 // 4. CloudTrail - Trail created but logging stopped
}
```

```
// 5-11. Other services: Lambda, SQS, SNS, SSM, Secrets Manager, DynamoDB, CloudWatch
}
```

## Running Provision Independently

You can run just the provisioning step without the full bootstrap:

```
npx tsx scripts/provision.ts
```

This is useful if you need to re-provision after resetting LocalStack.

## Re-provisioning

If you reset LocalStack and need to re-create the vulnerable resources:

```
Option 1: Run the full bootstrap
./scripts/bootstrap.sh

Option 2: Just re-provision
npx tsx scripts/provision.ts
```

Provisioning is **idempotent** — running it multiple times is safe. If a resource already exists, the script handles the conflict gracefully.

## 9. The 15 Vulnerable Resources — What Gets Created

### Complete Resource Inventory

After running `scripts/provision.ts`, LocalStack contains:

#### S3 (4 resources)

Resource	Name	Vulnerability	Severity
S3 Bucket	<code>vulnerable-bucket</code>	Public bucket policy (Principal: *)	 Critical
S3 Bucket	<code>vulnerable-bucket-logs</code>	Logging target (not vulnerable itself)	 Compliant
S3 Bucket	Missing Block Public Access	Inherited from bucket policy	 Critical
S3 Bucket	Missing Encryption	No SSE configured	 High
S3 Bucket	No Versioning	Versioning not enabled	 High
S3 Bucket	No Access Logging	Server access logs disabled	 Medium

## IAM (8 resources)

Resource	Name	Vulnerability	Severity
IAM User	<code>admin-user</code>	AdministratorAccess attached directly	 Critical
IAM User	<code>admin-user</code>	Old access keys (>90 days)	 Critical
IAM User	<code>stale-key-user</code>	One stale access key	 Medium
IAM User	<code>unused-key-user</code>	Never-used access keys	 Medium
IAM User	<code>unused-user</code>	No activity, no console login	 Medium
Password Policy	<code>account-password-policy</code>	Weak: min length 8, no symbols	 Medium
IAM User	Missing tags on some users	No Owner/Environment tags	 Low
IAM User	Missing descriptions	No description field	 Low

## EC2 (2 resources)

Resource	Name	Vulnerability	Severity
Security Group	<code>sg-vulnerable</code>	SSH port 22 open to 0.0.0.0/0	 High
Security Group	<code>sg-vulnerable</code>	RDP port 3389 open to 0.0.0.0/0	 High
Security Group	<code>sg-vulnerable</code>	No description	 Low

## CloudTrail (1 resource)

Resource	Name	Vulnerability	Severity
CloudTrail Trail	<code>mcpshield-trail</code>	Created but logging stopped	 High

## Additional Resources (5+ resources, compliant)

These are created for resource diversity — they are not vulnerable but help demonstrate the scanner's coverage:

- 1 Lambda function ( `vulnerable-function` )
- 1 SQS queue ( `vulnerable-queue` )
- 1 SNS topic ( `vulnerable-topic` )

- 1 SSM parameter ( `/mcpshield/vulnerable-param` )
- 1 Secrets Manager secret ( `vulnerable-secret` )
- 1 DynamoDB table ( `vulnerable-table` )
- 1 CloudWatch alarm ( `vulnerable-alarm` )

**Total: 15+ resources, 15 findings across 11 AWS services**

## Visual Map of the Vulnerable Environment

```
vulnerable-bucket (S3)
├─ ● Public access (Principal: "*")
├─ ● No encryption
├─ ● No versioning
├─ ● No access logging
├─ ● No resource tags
└─ ● Poor naming convention

admin-user (IAM)
├─ ● AdministratorAccess policy
├─ ● Old access keys (>90 days)
└─ ● Unused access keys

stale-key-user (IAM)
└─ ● Stale access key

unused-key-user (IAM)
└─ ● Never-used access key

unused-user (IAM)
└─ ● No activity for 90+ days

sg-vulnerable (EC2)
├─ ● SSH port 22 open to 0.0.0.0/0
├─ ● RDP port 3389 open to 0.0.0.0/0
└─ ● No description

mcpshield-trail (CloudTrail)
└─ ● Logging stopped

account-password-policy (IAM)
└─ ● Weak policy (min length 8)

other resources (Lambda, SQS, SNS, SSM, Secrets Manager, DynamoDB, CloudWatch)
└─ ✔ Compliant
```

## Why 15 Findings?

The 15 findings map to 15 MCPShield security rules:

#	Rule ID	Severity	Check
1	MCPS-S3-001	● Critical	Public S3 Bucket
2	MCPS-IAM-001	● Critical	AdministratorAccess on IAM User
3	MCPS-IAM-002	● Critical	Old Access Keys

4	MCPS-EC2-001	 High	SSH Open to Internet
5	MCPS-EC2-002	 High	RDP Open to Internet
6	MCPS-S3-002	 High	Missing S3 Encryption
7	MCPS-S3-003	 High	Missing S3 Versioning
8	MCPS-CT-001	 High	CloudTrail Disabled
9	MCPS-IAM-003	 Medium	Weak Password Policy
10	MCPS-IAM-004	 Medium	Unused IAM User
11	MCPS-IAM-005	 Medium	Unused Access Keys
12	MCPS-S3-004	 Medium	Missing Bucket Logging
13	MCPS-TAG-001	 Low	Missing Resource Tags
14	MCPS-NAM-001	 Low	Poor Naming Convention
15	MCPS-DESC-001	 Low	Missing Resource Descriptions

## 10. Manually Interacting with LocalStack

### Using AWS CLI with LocalStack

Every AWS CLI command needs the `--endpoint-url` flag to point to LocalStack:

```
General pattern
aws --endpoint-url=http://localhost:4566 <service> <command>

Shortcut: Set environment variable
export AWS_ENDPOINT_URL=http://localhost:4566
aws s3 ls
```

### S3 Operations

```
List buckets
aws --endpoint-url=http://localhost:4566 s3 ls

List bucket contents
aws --endpoint-url=http://localhost:4566 s3 ls s3://vulnerable-bucket

Check bucket policy
aws --endpoint-url=http://localhost:4566 s3api get-bucket-policy \
 --bucket vulnerable-bucket

Check public access block
aws --endpoint-url=http://localhost:4566 s3api get-public-access-block \
 --bucket vulnerable-bucket
```

```

Check encryption
aws --endpoint-url=http://localhost:4566 s3api get-bucket-encryption \
 --bucket vulnerable-bucket

Check versioning
aws --endpoint-url=http://localhost:4566 s3api get-bucket-versioning \
 --bucket vulnerable-bucket

Check logging
aws --endpoint-url=http://localhost:4566 s3api get-bucket-logging \
 --bucket vulnerable-bucket

Check tags
aws --endpoint-url=http://localhost:4566 s3api get-bucket-tagging \
 --bucket vulnerable-bucket

Create a new bucket
aws --endpoint-url=http://localhost:4566 s3 mb s3://my-new-bucket

Upload a file
echo "hello" > test.txt
aws --endpoint-url=http://localhost:4566 s3 cp test.txt s3://vulnerable-bucket/

Download a file
aws --endpoint-url=http://localhost:4566 s3 cp s3://vulnerable-bucket/test.txt .

```

## IAM Operations

```

List users
aws --endpoint-url=http://localhost:4566 iam list-users

List policies attached to a user
aws --endpoint-url=http://localhost:4566 iam list-attached-user-policies \
 --user-name admin-user

List access keys
aws --endpoint-url=http://localhost:4566 iam list-access-keys \
 --user-name admin-user

Get access key last used
aws --endpoint-url=http://localhost:4566 iam get-access-key-last-used \
 --access-key-id AKIA... (use actual key ID)

Check password policy
aws --endpoint-url=http://localhost:4566 iam get-account-password-policy

Detach policy (remediation action)
aws --endpoint-url=http://localhost:4566 iam detach-user-policy \
 --user-name admin-user \
 --policy-arn arn:aws:iam::aws:policy/AdministratorAccess

Deactivate access key (remediation action)
aws --endpoint-url=http://localhost:4566 iam update-access-key \
 --user-name admin-user \
 --access-key-id AKIA... \
 --status Inactive

Delete user (remediation action)
aws --endpoint-url=http://localhost:4566 iam delete-user \
 --user-name unused-user

```

## EC2 Operations

```
List security groups
aws --endpoint-url=http://localhost:4566 ec2 describe-security-groups

Check security group rules
aws --endpoint-url=http://localhost:4566 ec2 describe-security-groups \
 --group-ids sg-vulnerable (use actual SG ID)

Revoke SSH ingress (remediation action)
aws --endpoint-url=http://localhost:4566 ec2 revoke-security-group-ingress \
 --group-id sg-xxxxxx \
 --protocol tcp \
 --port 22 \
 --cidr 0.0.0.0/0
```

## CloudTrail Operations

```
List trails
aws --endpoint-url=http://localhost:4566 cloudtrail describe-trails

Check trail status
aws --endpoint-url=http://localhost:4566 cloudtrail get-trail-status \
 --name mcpshield-trail

Start logging (remediation action)
aws --endpoint-url=http://localhost:4566 cloudtrail start-logging \
 --name mcpshield-trail
```

## Other Services

```
Lambda
aws --endpoint-url=http://localhost:4566 lambda list-functions

SQS
aws --endpoint-url=http://localhost:4566 sqs list-queues

SNS
aws --endpoint-url=http://localhost:4566 sns list-topics

Secrets Manager
aws --endpoint-url=http://localhost:4566 secretsmanager list-secrets

SSM Parameter Store
aws --endpoint-url=http://localhost:4566 ssm describe-parameters

DynamoDB
aws --endpoint-url=http://localhost:4566 dynamodb list-tables

CloudWatch
aws --endpoint-url=http://localhost:4566 cloudwatch describe-alarms
```

---

## 11. How the Scanner Works

### Architecture

The scanner ( `packages/aws-tools/src/scanner.ts` ) is the heart of MCPShield's detection capability.

```
MCP Tool: scan_environment
|
▼
scanEnvironment()
|
├─ scanS3Buckets() → ListBuckets, GetPublicAccessBlock, GetBucketAcl, etc.
├─ scanIAM() → ListUsers, ListAttachedUserPolicies, GetAccessKeyLastUsed,
etc.
├─ scanEC2() → DescribeSecurityGroups
├─ scanCloudTrail() → DescribeTrails, GetTrailStatus
├─ scanLambda() → ListFunctions
├─ scanSQS() → ListQueues
├─ scanSNS() → ListTopics
├─ scanSecretsManager() → ListSecrets
├─ scanSSM() → DescribeParameters
├─ scanDynamoDB() → ListTables
└─ scanCloudWatch() → DescribeAlarms
|
▼
Array<ResourceSnapshot>
|
▼
Security Engine (runSecurityEngine)
|
├─ Rule 1: checkPublicS3Bucket
├─ Rule 2: checkAdminAccessAttached
├─ Rule 3: checkOldAccessKeys
├─ ...
└─ Rule 15: checkMissingDescriptions
|
▼
Array<Finding>
```

## How Each Service Is Scanned

### S3 Scanning (scanS3Buckets)

```
export async function scanS3Buckets(): Promise<ResourceSnapshot[]> {
 // 1. List all buckets
 const { Buckets } = await s3Client.send(new ListBucketsCommand({}));

 for (const bucket of Buckets) {
 // 2. For each bucket, gather security-relevant attributes:

 // Check 1: Public Access Block configuration
 try {
 const pab = await s3Client.send(
 new GetPublicAccessBlockCommand({ Bucket: bucket.Name })
);
 // Store pab.PublicAccessBlockConfiguration
 } catch {
 // No PublicAccessBlock = fully open
 }

 // Check 2: ACL grants (are there public grants?)
 const acl = await s3Client.send(
 new GetBucketAclCommand({ Bucket: bucket.Name })
);
 }
}
```

```

);
 // Check for AllUsers or AuthenticatedUsers grants

 // Check 3: Bucket Policy (are there public statements?)
 const policy = await s3Client.send(
 new GetBucketPolicyCommand({ Bucket: bucket.Name })
);
 // Check for Principal: "*" with Allow effect

 // Check 4: Encryption
 const enc = await s3Client.send(
 new GetBucketEncryptionCommand({ Bucket: bucket.Name })
);
 // Check if SSE is configured

 // Check 5: Versioning
 const ver = await s3Client.send(
 new GetBucketVersioningCommand({ Bucket: bucket.Name })
);
 // Check if Status === "Enabled"

 // Check 6: Logging
 const log = await s3Client.send(
 new GetBucketLoggingCommand({ Bucket: bucket.Name })
);
 // Check if LoggingEnabled is set

 // Check 7: Tags
 const tags = await s3Client.send(
 new GetBucketTaggingCommand({ Bucket: bucket.Name })
);
 // Check for Owner, Environment, DataClassification tags
}
}

```

## IAM Scanning (scanIAM)

```

export async function scanIAM(): Promise<ResourceSnapshot[]> {
 // 1. Get account password policy
 const policyRes = await iamClient.send(
 new GetAccountPasswordPolicyCommand({})
);

 // 2. List all users
 const usersRes = await iamClient.send(new ListUsersCommand({}));

 for (const user of usersRes.Users) {
 // 3. For each user, check:

 // Check A: Attached policies
 const policies = await iamClient.send(
 new ListAttachedUserPoliciesCommand({ UserName: user.UserName })
);
 // Is AdministratorAccess attached?

 // Check B: Access keys
 const keysRes = await iamClient.send(
 new ListAccessKeysCommand({ UserName: user.UserName })
);
 for (const key of keysRes.AccessKeyMetadata) {
 // Check key age (>90 days?)
 // Check last usage (>90 days ago?)
 }

 // Check C: Console login
 }
}

```

```

try {
 await iamClient.send(
 new GetLoginProfileCommand({ UserName: user.UserName })
);
 // User has console access
} catch {
 // No console access
}

// Check D: Tags
const tagsRes = await iamClient.send(
 new ListUserTagsCommand({ UserName: user.UserName })
);
}
}

```

## What Each Scan Call Returns

A `ResourceSnapshot` looks like:

```

interface ResourceSnapshot {
 service: string; // e.g., "s3", "iam", "ec2"
 type: string; // e.g., "bucket", "user", "security-group"
 id: string; // Resource identifier
 arn?: string; // AWS ARN
 region: string; // AWS region
 attributes: Record<string, unknown>; // All gathered data
 tags: Record<string, string>; // Resource tags
}

```

Example S3 snapshot:

```

{
 "service": "s3",
 "type": "bucket",
 "id": "vulnerable-bucket",
 "arn": "arn:aws:s3:::vulnerable-bucket",
 "region": "us-east-1",
 "attributes": {
 "creationDate": "2026-07-14T12:00:00.000Z",
 "publicAccessBlock": null,
 "acl": {
 "grants": [
 {
 "Grantee": { "URI": "http://acs.amazonaws.com/groups/global/AllUsers" },
 "Permission": "READ"
 }
]
 },
 "policy": {
 "Version": "2012-10-17",
 "Statement": [
 {
 "Effect": "Allow",
 "Principal": "*",
 "Action": "s3:GetObject",
 "Resource": "arn:aws:s3:::vulnerable-bucket/*"
 }
]
 },
 "encryption": null,
 "versioning": { "Status": "Suspended" },
 }
}

```

```

 "logging": null,
 "tags": {}
 },
 "tags": {}
}

```

## 12. How Remediation Works

### Remediation Flow

When a user approves remediation via `@Shield approve` :

```

User types: @Shield approve
|
▼
MCP Tool: execute_remediation
|
▼
mapFindingToRemediation() ← Maps finding to RemediationAction
|
▼
executeRemediationAction() ← In aws-tools/remediator.ts
|
▼
AWS SDK call to modify LocalStack resource
|
▼
LocalStack processes the change
|
▼
Re-scan confirms fix

```

### How the Remediation Mapper Works

The function `mapFindingToRemediation()` in `apps/mcp-server/src/server.ts` maps each finding type to a specific remediation action:

```

function mapFindingToRemediation(finding: Finding): RemediationAction | null {
 switch (finding.catalogId) {
 case 'MCPS-S3-001':
 return {
 operation: 'putPublicAccessBlock',
 params: { bucket: resourceId }
 };

 case 'MCPS-IAM-001':
 return {
 operation: 'detachUserPolicy',
 params: {
 userName: resourceId,
 policyArn: 'arn:aws:iam::aws:policy/AdministratorAccess'
 }
 };

 case 'MCPS-IAM-002':
 return {

```

```

 operation: 'deactivateAccessKey',
 params: { userName: resourceId, accessKeyId }
 };

 case 'MCPS-EC2-001':
 return {
 operation: 'revokeSecurityGroupIngress',
 params: { sgId: resourceId, port: 22 }
 };

 // ... 9 more operations
}

```

## How Remediation Executes

The `executeRemediationAction()` function in `packages/aws-tools/src/remediator.ts` performs the actual AWS SDK call:

```

export async function executeRemediationAction(
 action: RemediationAction
): Promise<RemediationResult> {
 switch (action.operation) {
 case 'putPublicAccessBlock':
 await s3Client.send(new PutPublicAccessBlockCommand({
 Bucket: action.params.bucket,
 PublicAccessBlockConfiguration: {
 BlockPublicAcls: true,
 IgnorePublicAcls: true,
 BlockPublicPolicy: true,
 RestrictPublicBuckets: true,
 },
 }));
 break;

 case 'detachUserPolicy':
 await iamClient.send(new DetachUserPolicyCommand({
 UserName: action.params.userName,
 PolicyArn: action.params.policyArn,
 }));
 break;

 // ... 7 more operations
 }

 return { success: true, message: 'Remediation applied successfully' };
}

```

## Supported Remediation Operations

Finding	Operation	What It Does to LocalStack
MCPS-S3-001	<code>putPublicAccessBlock</code>	Enables Block Public Access on S3 bucket
MCPS-IAM-001	<code>detachUserPolicy</code>	Removes AdministratorAccess from user
MCPS-IAM-002	<code>deactivateAccessKey</code>	Marks stale key as Inactive

MCPS-EC2-001	<code>revokeSecurityGroupIngress</code>	Removes SSH 0.0.0.0/0 rule
MCPS-EC2-002	<code>revokeSecurityGroupIngress</code>	Removes RDP 0.0.0.0/0 rule
MCPS-S3-002	<code>putBucketEncryption</code>	Enables AES256 encryption on bucket
MCPS-S3-003	<code>putBucketVersioning</code>	Enables versioning on bucket
MCPS-CT-001	<code>startLogging</code>	Starts CloudTrail logging
MCPS-IAM-003	<code>updatePasswordPolicy</code>	Sets strong password policy
MCPS-IAM-004	<code>deleteUser</code>	Deletes unused IAM user
MCPS-IAM-005	<code>deactivateAccessKey</code>	Deactivates unused key
MCPS-S3-004	<code>putBucketLogging</code>	Enables access logging
MCPS-TAG-001	<code>putBucketTagging</code>	Adds required tags to bucket
MCPS-NAM-001	<code>renameBucket</code>	Cannot rename — creates new bucket
MCPS-DESC-001	<code>putParameterDescription</code>	Adds description to SSM parameter

## What Happens in LocalStack After Remediation

Each remediation modifies the state in LocalStack:

### Before:

```
S3 Bucket: vulnerable-bucket
Policy: Principal: "*" (public)
Encryption: None
Versioning: Suspended
```

### After putPublicAccessBlock:

```
S3 Bucket: vulnerable-bucket
Policy: Principal: "*" (still public)
But: BlockPublicAcls=true, BlockPublicPolicy=true → Policy is now blocked
```

### After putBucketEncryption:

```
S3 Bucket: vulnerable-bucket
Policy: Principal: "*" (still public)
BlockPublicAcls: true
Encryption: AES256 ✓
Versioning: Suspended
```

## Verifying Remediation

After remediation, MCPSHield re-scans the specific resource to verify:

```
// In rescan_environment tool handler
const snapshots = await scanEnvironment();
const findings = runSecurityEngine(snapshots);
// Compare new findings with old findings
// If finding is gone → remediation confirmed
```

---

## 13. Common LocalStack Commands

### Lifecycle Commands

```
Start LocalStack (background)
localstack start -d

Start LocalStack (foreground, for logs)
localstack start

Stop LocalStack
localstack stop

Restart LocalStack
localstack restart

Check status
localstack status

Show logs
localstack logs

Show logs for a specific service
localstack logs --service s3
```

### State Management

```
Save state to disk (if persistence is enabled)
curl -X POST http://localhost:4566/_localstack/state/save

Load state from disk
curl -X POST http://localhost:4566/_localstack/state/load

Reset all state (clear everything)
localstack stop
localstack start -d
```

### Configuration

```
List enabled services
curl -s http://localhost:4566/_localstack/health | python3 -c "import sys,json;
print(json.load(sys.stdin)['services'].keys())"

Get LocalStack configuration
curl -s http://localhost:4566/_localstack/config | python3 -m json.tool
```

```
Update configuration at runtime
curl -X POST http://localhost:4566/_localstack/config \
 -H "Content-Type: application/json" \
 -d '{"SERVICES": "s3,iam,ec2"}'
```

## Debugging

```
Enable debug logging
export LOCALSTACK_LOG_LEVEL=debug
localstack start -d

Check memory usage
curl -s http://localhost:4566/_localstack/health | python3 -c "
import sys, json
h = json.load(sys.stdin)
print('Status:', h['status'])
print('Services available:', sum(1 for s in h['services'].values() if s == 'available'))
for svc, status in h['services'].items():
 print(f' {svc}: {status}')
"

Network connectivity test
curl -v http://localhost:4566/ 2>&1 | head -20
```

## Cleanup

```
Remove all data in a specific service
aws --endpoint-url=http://localhost:4566 s3 rb s3://vulnerable-bucket --force

Remove all LocalStack data
localstack stop
rm -rf ~/.cache/localstack # Linux/macOS
or
rm -rf %USERPROFILE%\cache\localstack # Windows
```

## Docker-Specific Commands

```
Start LocalStack in Docker with specific services
SERVICES=s3,iam,ec2 localstack start -d

Start with persistence
localstack start -d --persist

View Docker logs
docker logs localstack-main

Access Docker container shell
docker exec -it localstack-main bash
```

---

## 14. Troubleshooting LocalStack

## LocalStack Won't Start

Symptom	Cause	Solution
Error: Port 4566 is already in use	Another process is using the port	<code>lsof -i :4566</code> → <code>kill -9 &lt;PID&gt;</code>
Error: Docker daemon not running	LocalStack in Docker mode	<code>open -a Docker</code> (macOS) or <code>sudo systemctl start docker</code> (Linux)
Error: Python version not supported	Python < 3.9 installed	<code>python3 --version</code> → upgrade to 3.9+
Error: Failed to start services	Resource conflict	<code>localstack stop</code> → <code>localstack start -d</code>
Starts but immediately stops	Memory issue	<code>free -m</code> → check available memory (need 2GB+)

## LocalStack Starts But Health Check Fails

Symptom	Cause	Solution
<code>curl: (7) Failed to connect</code>	LocalStack not listening	Wait 10 seconds, retry
<code>status: "starting"</code>	Still initializing	Wait 15 seconds, retry
<code>status: "error"</code>	Service failed to start	<code>localstack logs</code> → check error
Certain services show stopped	Services not enabled	<code>export SERVICES=s3,iam,ec2,...</code> → restart

## AWS CLI Commands Fail

Symptom	Cause	Solution
Could not connect to endpoint	Wrong endpoint URL	Use <code>--endpoint-url=http://localhost:4566</code>
SignatureDoesNotMatch	Auth header mismatch	LocalStack accepts any credentials
AccessDenied	IAM policy enforcement	LocalStack doesn't enforce IAM (should not happen)
NoSuchBucket	Bucket doesn't exist	Run <code>./scripts/provision.ts</code> first

<code>ResourceNotFoundException</code>	Resource not provisioned	Check provisioning script ran successfully
----------------------------------------	--------------------------	--------------------------------------------

## Provisioning Fails

Symptom	Cause	Solution
<code>EntityAlreadyExists</code>	Resource already exists	This is normal — provisioning is idempotent
<code>LocalStack is not reachable</code>	LocalStack not running	<code>localstack start -d</code>
<code>Failed to create bucket</code>	LocalStack S3 not ready	Wait and retry: <code>npx tsx scripts/provision.ts</code>
Script times out	LocalStack overwhelmed	Start with fewer services: <code>SERVICES=s3,iam,ec2 localstack start -d</code>

## Performance Issues

Symptom	Cause	Solution
API calls take >1 second	Machine underpowered	Close other apps, ensure 4GB+ RAM
LocalStack uses 100% CPU	Memory pressure	Restart LocalStack
Docker builds slow	No cache	Pre-pull images: <code>docker pull node:22-alpine</code>
Scan returns partial results	Timeout	<code>export AWS_TIMEOUT=10000</code> (increase timeout)

## Network Issues

Symptom	Cause	Solution
Can't connect from Docker containers	Wrong hostname	Use <code>http://host.docker.internal:4566</code>
Can't connect from WSL2	Linux subsystem	Use Powershell: <code>curl http://localhost:4566</code>
Connection refused after restart	Port not released	Wait 5 seconds, check <code>lsof -i :4566</code>

## Complete Reset Procedure

If LocalStack is in a bad state:

```
1. Stop LocalStack
localstack stop

2. Verify it's stopped
curl http://localhost:4566 # Should fail

3. Clear LocalStack cache
rm -rf ~/.cache/localstack

4. Verify port is free
lsof -i :4566 # Should show nothing

5. Start fresh
localstack start -d

6. Verify health
curl http://localhost:4566/_localstack/health

7. Re-provision
cd ~/mcpshield
npx tsx scripts/provision.ts

8. Verify resources
aws --endpoint-url=http://localhost:4566 s3 ls
aws --endpoint-url=http://localhost:4566 iam list-users
```

This entire reset takes less than 30 seconds.

---

## 15. Advanced: Custom LocalStack Configuration

### Environment Variables

LocalStack can be configured with environment variables:

```
Configure services (comma-separated)
export SERVICES=s3,iam,ec2,cloudtrail,lambda,sqs,sns,secretsmanager,ssm,dynamodb,cloudwatch

Configure port
export EDGE_PORT=4566

Enable persistence (save state to disk)
export PERSISTENCE=1

Set debug logging
export LS_LOG=trace

Configure host
export LOCALSTACK_HOST=localhost

Start with custom config
localstack start -d
```

### Using a Configuration File

Create `~/.localstack/config.json` :

```
{
 "services":
 "s3,iam,ec2,cloudtrail,lambda,sqs,sns,secretsmanager,ssm,dynamodb,cloudwatch",
 "edge_port": 4566,
 "persistence": false,
 "debug": false,
 "host": "localhost"
}
```

## Docker Compose with LocalStack

You can run LocalStack in Docker Compose alongside MCPShield:

```
services:
 localstack:
 image: localstack/localstack:latest
 ports:
 - "4566:4566"
 environment:
 - SERVICES=s3,iam,ec2,cloudtrail,lambda,sqs,sns,secretsmanager,ssm,dynamodb,cloudwatch
 - PERSISTENCE=0
 volumes:
 - localstack-data:/var/lib/localstack

 mcp-server:
 build: ./docker/mcp-server
 environment:
 - LOCALSTACK_ENDPOINT=http://localstack:4566
 depends_on:
 - localstack
```

**Note:** For the workshop, LocalStack runs on the host machine (not in Docker Compose) so that participants can interact with it directly via AWS CLI and see what's happening.

---

## 16. Advanced: Multi-Region Setup

### Why Multi-Region Matters

CloudTrail findings check for multi-region trails. LocalStack supports multiple regions, allowing you to test cross-region security configurations.

### Configuring Additional Regions

```
Start LocalStack with default region
export AWS_DEFAULT_REGION=us-east-1
localstack start -d

Create resources in different regions
aws --endpoint-url=http://localhost:4566 --region us-east-1 s3 mb s3://us-east-bucket
```

```
aws --endpoint-url=http://localhost:4566 --region eu-west-1 s3 mb s3://eu-west-bucket
aws --endpoint-url=http://localhost:4566 --region ap-southeast-1 s3 mb s3://ap-southeast-bucket
```

## How the Scanner Handles Regions

In `packages/aws-tools/src/clients.ts`, the AWS SDK clients are configured with the default region:

```
export function getClientConfig() {
 const config = getConfig();
 return {
 endpoint: config.aws.endpoint,
 region: config.aws.region, // From .env: AWS_DEFAULT_REGION
 };
}
```

For the workshop, all resources are in `us-east-1`. The multi-region CloudTrail finding checks if trails span all regions.

---

## 17. Advanced: Persistent State

### Enabling Persistence

By default, LocalStack loses all data when restarted. To keep data between restarts:

```
Enable persistence
export PERSISTENCE=1
localstack start -d
```

Or create `~/.localstack/config.json`:

```
{
 "persistence": true
}
```

### Where State Is Stored

- **Linux/macOS:** `~/.cache/localstack/`
- **Windows:** `%USERPROFILE%\cache\localstack\`
- **Docker:** `/var/lib/localstack` (inside container)

### State Storage Structure

```
~/.cache/localstack/
├─ state.json # All service states
```

```
└─ s3/ # S3 object data
 └─ vulnerable-bucket/
 └─ test.txt
└─ lambda/ # Lambda function code
 └─ tmp/
```

## Persistence in the Workshop

For the workshop, persistence is **not required** because:

1. The bootstrap script creates resources fresh each time
2. Participants may want to reset their environment
3. No persistent data is needed between sessions

## 18. Advanced: Lambda Functions in LocalStack

## How Lambda Works in LocalStack

LocalStack can execute Lambda functions locally. The bootstrap script creates a Lambda function for resource diversity:

```
// In scripts/provision.ts
const EMPTY_ZIP_BYTES = new Uint8Array([
 80, 75, 5, 6, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
]);

await lambdaClient.send(new CreateFunctionCommand({
 FunctionName: 'vulnerable-function',
 Runtime: 'nodejs18.x',
 Handler: 'index.handler',
 Role: 'arn:aws:iam::000000000000:role/lambda-role',
 Code: { ZipFile: EMPTY_ZIP_BYTES },
})));
```

## Invoking Lambda Functions

```
aws --endpoint-url=http://localhost:4566 lambda invoke \
 --function-name vulnerable-function \
 --payload '{}' \
 output.json
```

## Lambda Limitations in LocalStack

- Execution timeout is configurable
- Some runtime versions may not be available
- No VPC networking support (free version)
- Limited CloudWatch Logs integration

## 19. Resetting the Environment

### Quick Reset (Recommended)

This is the fastest way to reset for the workshop:

```
1. Stop LocalStack (clears all state)
localstack stop

2. Start fresh
localstack start -d

3. Wait for readiness
sleep 5
curl http://localhost:4566/_localstack/health

4. Re-provision
cd ~/mcpshield
npx tsx scripts/provision.ts
```

**Total time:** ~15 seconds

### Full Reset (Clean Slate)

This removes all cached data:

```
1. Stop LocalStack and MCPShield
localstack stop
docker compose down

2. Clear all state
rm -rf ~/.cache/localstack
rm -rf ~/mcpshield/.mcpshield-state

3. Start fresh
localstack start -d
sleep 5

4. Re-bootstrap
cd ~/mcpshield
./scripts/bootstrap.sh

5. Start services
docker compose up --build -d
```

**Total time:** ~2 minutes

### Selective Reset (Single Service)

Reset just one service without affecting others:

```
Reset only S3
aws --endpoint-url=http://localhost:4566 s3 rb s3://vulnerable-bucket --force
aws --endpoint-url=http://localhost:4566 s3 rb s3://vulnerable-bucket-logs --force
```

```
Re-create just the S3 resources (you'd need to add this to provision.ts)
```

## When to Reset

Scenario	Reset Type	Why
Before workshop	Quick reset	Fresh environment for participants
After testing remediation	Quick reset	Undo changes for next demo
LocalStack behaving strangely	Full reset	Clear corrupted state
Between workshop sessions	Quick reset	New environment for next group
Bootstrap fails repeatedly	Full reset	Clear all cached state

## 20. Frequently Asked Questions

### General

#### Q: Does LocalStack require Docker?

A: No. LocalStack can run directly with `pip install localstack`. Docker is optional but recommended for production use.

#### Q: Is LocalStack free?

A: Yes, the Community Edition used in this workshop is free and open-source.

#### Q: Can I use LocalStack for production?

A: No. LocalStack is for development and testing. Use real AWS for production.

#### Q: Does LocalStack support all AWS services?

A: No. It supports ~15 services in the free version. The Pro version adds more.

### Workshop-Specific

#### Q: Why doesn't Docker Compose start LocalStack?

A: By design. LocalStack runs on the host machine so participants can interact with it directly via AWS CLI. Docker containers connect via `host.docker.internal`.

#### Q: Can I share my LocalStack environment with others?

A: Not directly. Each participant runs LocalStack locally. This is intentional — everyone gets their own isolated environment.

#### Q: The health check shows a service as "stopped"

A: Restart LocalStack with the full service list:

```
SERVICES=s3,iam,ec2,cloudtrail,lambda,sqs,sns,secretsmanager,ssm,dynamodb,cloudwatch
localstack start -d
```

**Q: Can I use LocalStack without the bootstrap script?**

A: Yes. But you'll need to create resources manually. The bootstrap script sets up the intentionally vulnerable environment needed for the workshop.

## Technical

**Q: How does LocalStack handle credentials?**

A: It accepts any credentials. IAM policies are not enforced in the Community Edition. This is by design for development/testing.

**Q: Is data persisted across restarts?**

A: No, by default. All data is in-memory and lost when LocalStack stops. This is intentional for the workshop.

**Q: Can I run LocalStack on a different port?**

A: Yes. Set `EDGE_PORT=4567` and update `LOCALSTACK_ENDPOINT` in your `.env`.

**Q: What's the maximum number of resources I can create?**

A: LocalStack has no artificial limits. You can create thousands of resources (within your machine's memory limits).

**Q: Can I run LocalStack on a Raspberry Pi?**

A: Yes, but performance will be limited. The recommended configuration is 4GB+ RAM.

---

## 21. Quick Reference

### Essential Commands

```
Start/Stop
localstack start -d # Start in background
localstack stop # Stop
localstack restart # Restart

Health Check
curl http://localhost:4566/_localstack/health

Basic AWS CLI Tests
aws --endpoint-url=http://localhost:4566 s3 ls
aws --endpoint-url=http://localhost:4566 iam list-users
aws --endpoint-url=http://localhost:4566 ec2 describe-security-groups

Provision
npx tsx scripts/provision.ts

Reset
localstack stop && localstack start -d && sleep 5 && npx tsx scripts/provision.ts

View Logs
localstack logs --follow
```

### Key Endpoints

Service	LocalStack Endpoint
API Gateway	<code>http://localhost:4566</code>
S3	<code>http://localhost:4566</code>
IAM	<code>http://localhost:4566</code>
EC2	<code>http://localhost:4566</code>
CloudTrail	<code>http://localhost:4566</code>
Lambda	<code>http://localhost:4566</code>
SQS	<code>http://localhost:4566</code>
SNS	<code>http://localhost:4566</code>
Secrets Manager	<code>http://localhost:4566</code>
SSM	<code>http://localhost:4566</code>
DynamoDB	<code>http://localhost:4566</code>
CloudWatch	<code>http://localhost:4566</code>
Health/Status	<code>http://localhost:4566/_localstack/health</code>

## Environment Variables

Variable	Default	Description
<code>LOCALSTACK_ENDPOINT</code>	<code>http://localhost:4566</code>	MCPShield's link to LocalStack
<code>AWS_ACCESS_KEY_ID</code>	<code>test</code>	Dummy credential
<code>AWS_SECRET_ACCESS_KEY</code>	<code>test</code>	Dummy credential
<code>AWS_DEFAULT_REGION</code>	<code>us-east-1</code>	Default region

## MCPShield ↔ LocalStack Integration Points

MCPShield Package	LocalStack Service	Key Operations
<code>packages/aws-tools/src/clients.ts</code>	All	Client initialization
<code>packages/aws-tools/src/scanner.ts</code>	S3, IAM, EC2, CloudTrail, Lambda, SQS, SNS, Secrets Manager, SSM, DynamoDB, CloudWatch	Resource scanning

<code>packages/aws-</code> <code>tools/src/remediator.ts</code>	S3, IAM, EC2, CloudTrail, SSM	Remediation execution
<code>scripts/provision.ts</code>	All	Resource provisioning
<code>apps/mcp-</code> <code>server/src/server.ts</code>	S3 (health check)	LocalStack connectivity check

## Workshop Flow with LocalStack

Step 1: <code>install localstack</code>	← Before workshop
Step 2: <code>localstack start -d</code>	← Start mock AWS
Step 3: <code>./scripts/bootstrap.sh</code>	← Build + provision
Step 4: <code>docker compose up</code>	← Start MCPShield
Step 5: <code>@Shield scan environment</code>	← Scan mock AWS
Step 6: <code>@Shield fix finding &lt;id&gt;</code>	← Fix a finding
Step 7: <code>@Shield approve</code>	← Execute in LocalStack
Step 8: <code>@Shield rescan</code>	← Verify fix
Step 9: <code>@Shield generate report</code>	← Generate assessment

*This guide was prepared for the Build with AI OAU 2026 Cybersecurity Breakout Session.*

*Speaker: Olumayowa Akinkuehinmi | Repository: <https://github.com/akintunero/mcpshield>*